

LOAN APPROVAL CLASSIFICATION

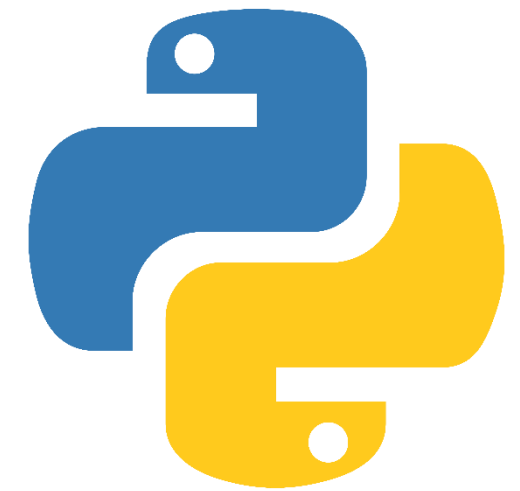
● RAUDAH YASMIN GHOZALI

FEBRUARI 2025

TOOLS



kaggle



DESKRIPSI VARIABEL

Variabel	Deskripsi
person_age	Age of the person
person_gender	Gender of the person
person_education	Highest education level
person_income	Annual income
person_emp_exp	Years of employment experience
person_home_ownership	Home ownership status
loan_amnt	Loan amount requested
loan_intent	Purpose of the loan
loan_int_rate	Loan interest rate
loan_percent_income	Loan amount as a percentage of annual income
cb_person_cred_hist_length	Length of credit history in years
credit_score	Credit score of the person
previous_loan_defaults_on_file	Indicator of previous loan defaults
loan_status (target variable)	Loan approval status

TA-WEI LO · UPDATED 4 MONTHS AGO

▲123

<> Code

Download

Loan Approval Classification Dataset

Synthetic Data for binary classification on Loan Approval

Data Card

Code (97)

Discussion (2)

Suggestions (0)

About Dataset

1. Data Source

This dataset is a synthetic version inspired by the original Credit Risk dataset on Kaggle and enriched with additional variables based on [Financial Risk for Loan Approval data](#). SMOTENC was used to simulate new data points to enlarge the instances. The dataset is structured for both categorical and continuous features.

Usability ⓘ

10.00

License

[Apache 2.0](#)

Expected update frequency

Never



Link dataset:
<https://www.kaggle.com/datasets/taweilo/loan-approval-classification-data/data>

IMPORT DATA

Impor data dari device dan diupload ke Google Colab. Setelah itu, untuk memunculkan data menggunakan pandas dan untuk membaca file disesuaikan dengan format file.



```
#Impor data dari device
from google.colab import files
files.upload()
```



Show hidden output



```
#Membaca data
import pandas as pd
data=pd.read_csv('loan_data.csv')
data
```

	person_age	person_gender	person_education	person_income	person_emp_exp	person_home_ownership	loan_amnt	loan_intent	loan_int_rate	loan_percent_income	cb_person_cred_hist_length	credit_score
0	22.0	female	Master	71948.0	0	RENT	35000.0	PERSONAL	16.02	0.49	3.0	5
1	21.0	female	High School	12282.0	0	OWN	1000.0	EDUCATION	11.14	0.08	2.0	5
2	25.0	female	High School	12438.0	3	MORTGAGE	5500.0	MEDICAL	12.87	0.44	3.0	6
3	23.0	female	Bachelor	79753.0	0	RENT	35000.0	MEDICAL	15.23	0.44	2.0	6
4	24.0	male	Master	66135.0	1	RENT	35000.0	MEDICAL	14.27	0.53	4.0	5
...	...	...	...	...	...	...	...	...	...	...	...	...
44995	27.0	male	Associate	47971.0	6	RENT	15000.0	MEDICAL	15.66	0.31	3.0	6
44996	37.0	female	Associate	65800.0	17	RENT	9000.0	HOMEIMPROVEMENT	14.07	0.14	11.0	6
44997	33.0	male	Associate	56942.0	7	RENT	2771.0	DEBTCONSOLIDATION	10.02	0.05	10.0	6
44998	29.0	male	Bachelor	33164.0	4	RENT	12000.0	EDUCATION	13.23	0.36	6.0	6
44999	24.0	male	High School	51609.0	1	RENT	6665.0	DEBTCONSOLIDATION	17.05	0.13	3.0	6

45000 rows × 14 columns

EXPLORATORY DATA ANALYSIS (EDA)



Informasi Data

```
#Informasi data (jumlah kolom, baris, tipe data)
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 45000 entries, 0 to 44999
Data columns (total 14 columns):
 #   Column                                  Non-Null Count  Dtype  
---  -
 0   person_age                             45000 non-null  float64
 1   person_gender                           45000 non-null  object  
 2   person_education                         45000 non-null  object  
 3   person_income                           45000 non-null  float64
 4   person_emp_exp                           45000 non-null  int64   
 5   person_home_ownership                   45000 non-null  object  
 6   loan_amnt                               45000 non-null  float64
 7   loan_intent                             45000 non-null  object  
 8   loan_int_rate                           45000 non-null  float64
 9   loan_percent_income                     45000 non-null  float64
10   cb_person_cred_hist_length              45000 non-null  float64
11   credit_score                             45000 non-null  int64   
12   previous_loan_defaults_on_file          45000 non-null  object  
13   loan_status                             45000 non-null  int64   
dtypes: float64(6), int64(3), object(5)
memory usage: 4.8+ MB
```

Informasi data digunakan untuk mengetahui berapa banyak entri dan variabel pada dataset, dan tipe data setiap variabel. Informasi ini dapat digunakan untuk langkah preprocessing data. Dapat dilihat dataset memiliki 45000 entri dengan variabel sebanyak 14 kolom.

EXPLORATORY DATA ANALYSIS (EDA)



Bar Charts

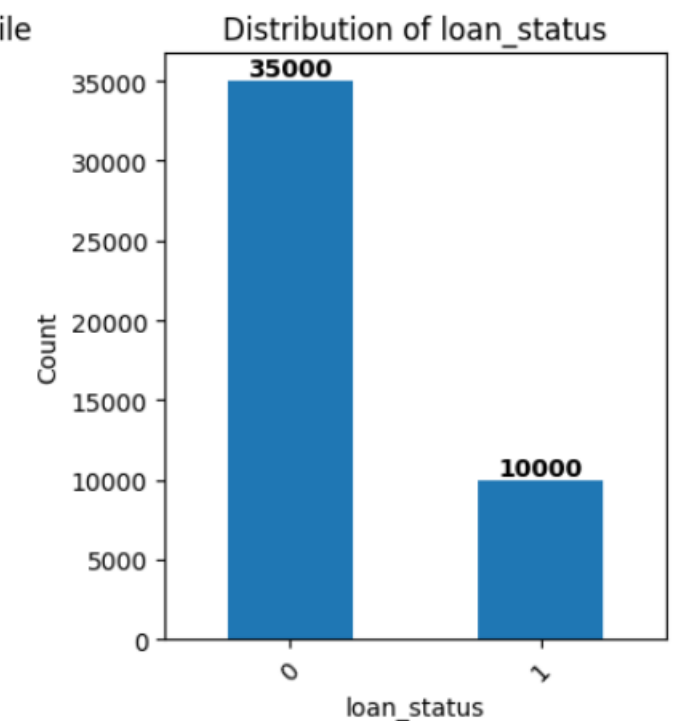
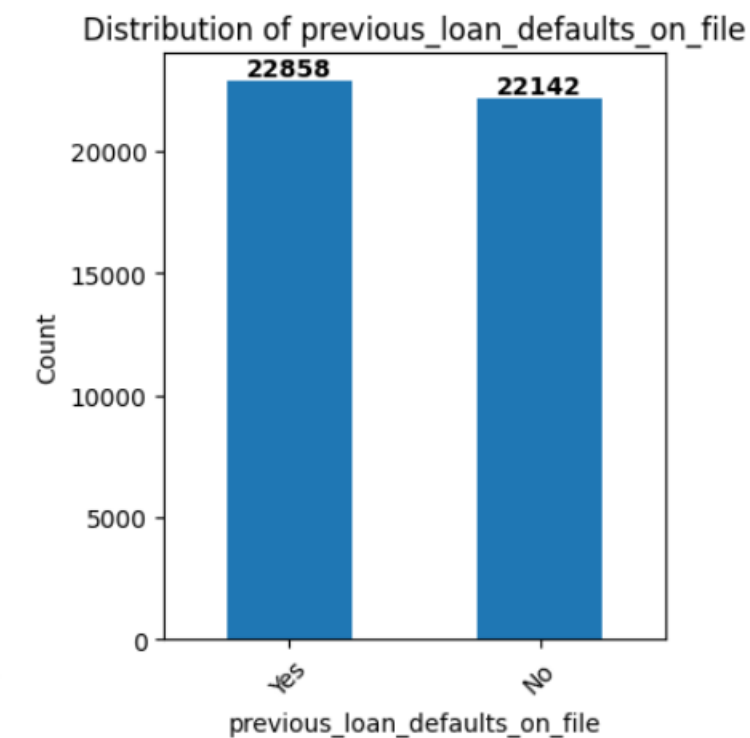
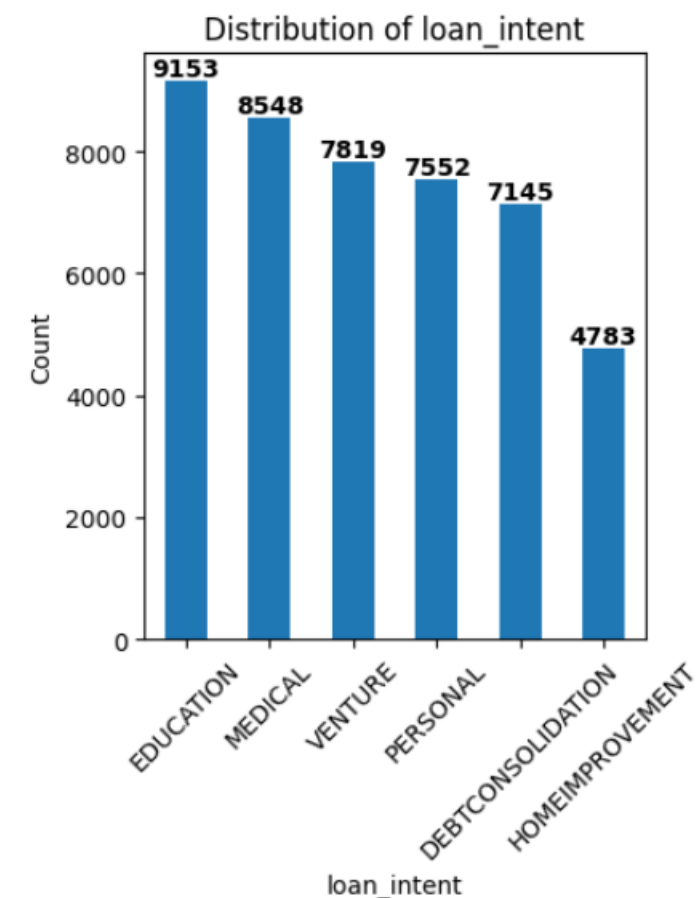
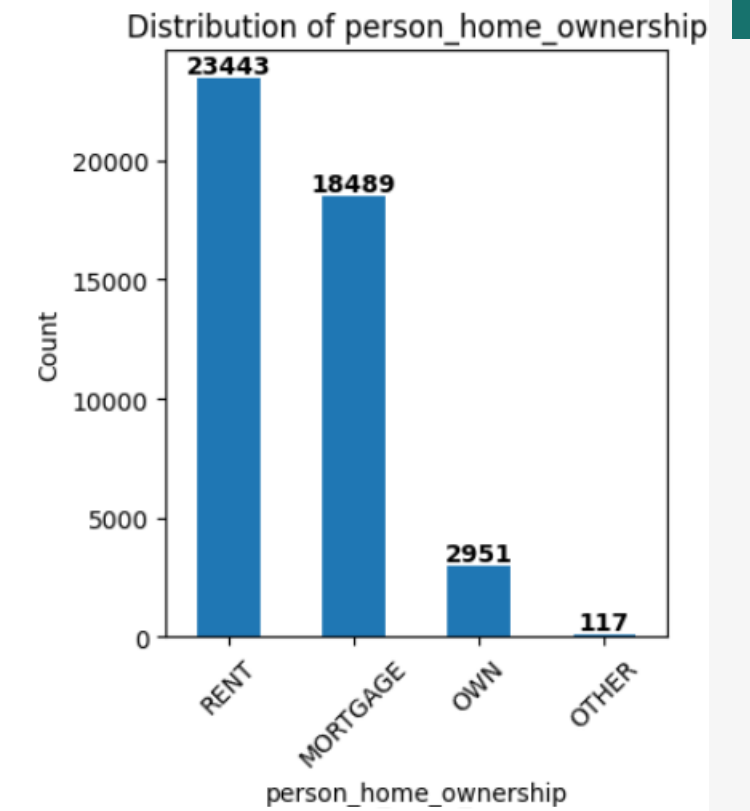
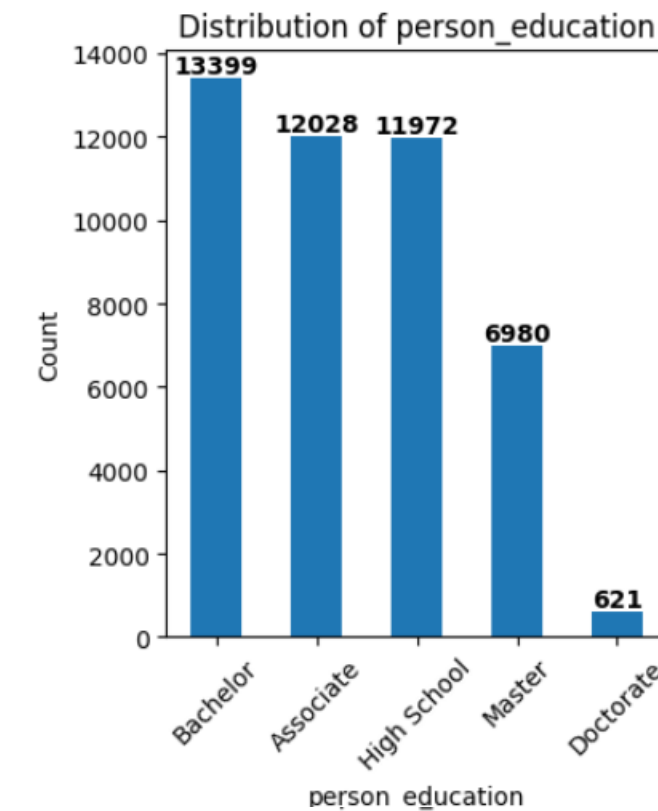
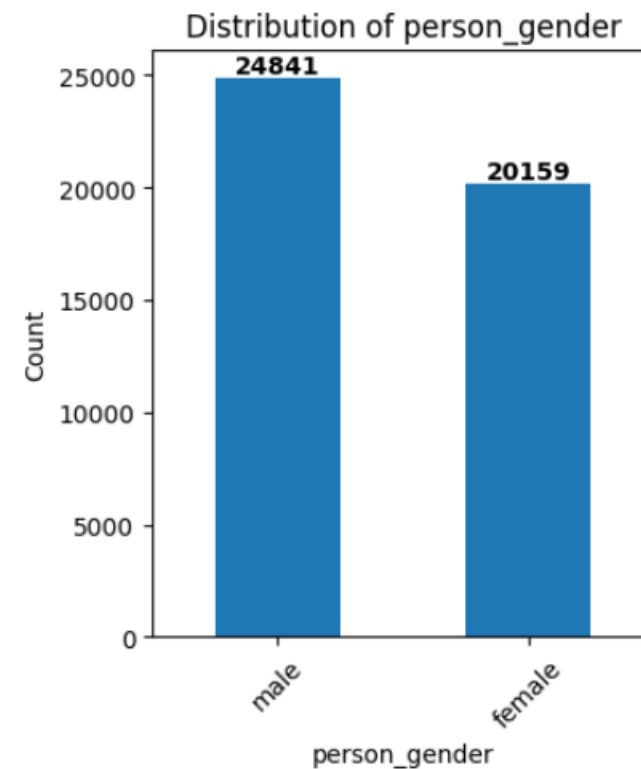
```
#Visualisasi distribusi data dengan bar chat
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 3, figsize=(12, 10)) #Membuat subplot (baris, kolom, ukuran plot)
features = ['person_gender', 'person_education', 'person_home_ownership',
            'loan_intent', 'previous_loan_defaults_on_file', 'loan_status']

#Perulangan untuk plotting setiap fitur
for ax, feature in zip(axes.flatten(), features):
    value_counts = data[feature].value_counts()
    bars = value_counts.plot(kind='bar', ax=ax)
    ax.set_title(f"Distribution of {feature}")
    ax.set_ylabel("Count")
    ax.tick_params(axis='x', rotation=45)

    for bar in bars.patches:
        ax.text(bar.get_x() + bar.get_width()/2,
                bar.get_height(),
                str(int(bar.get_height())),
                ha='center',
                va='bottom',
                fontsize=10,
                color='black',
                fontweight='bold')

plt.tight_layout(rect=[0, 0, 1, 1])
plt.show()
```

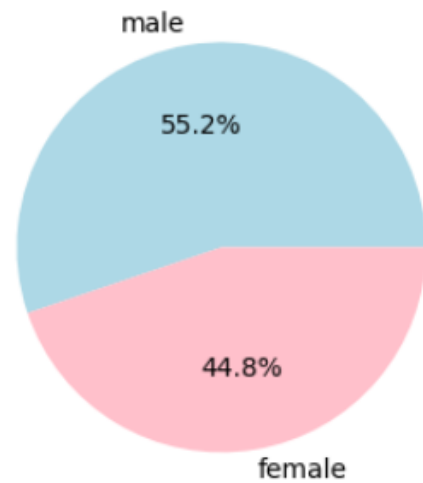


EXPLORATORY DATA ANALYSIS (EDA)

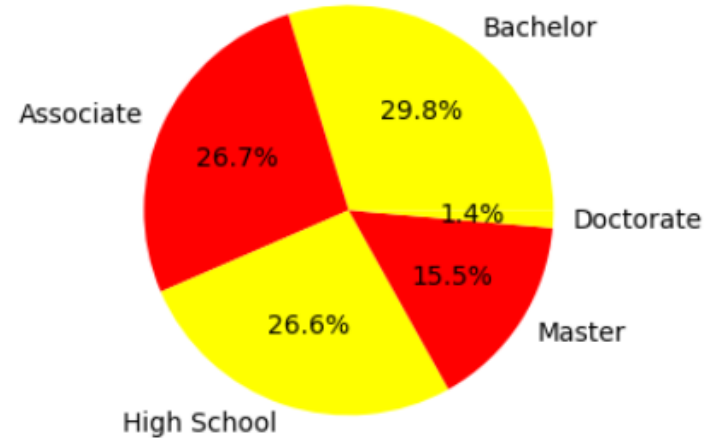


Pie Charts

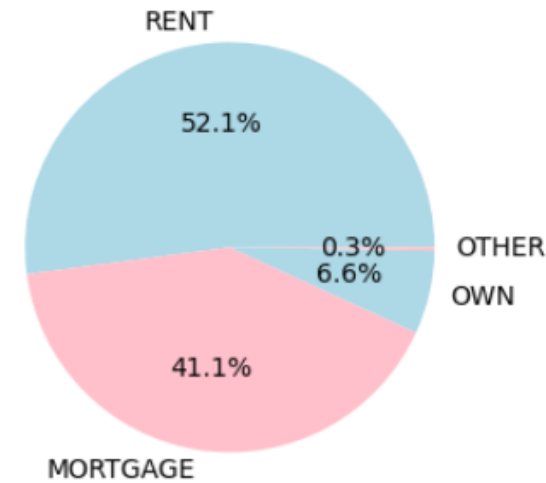
Distribution of Gender



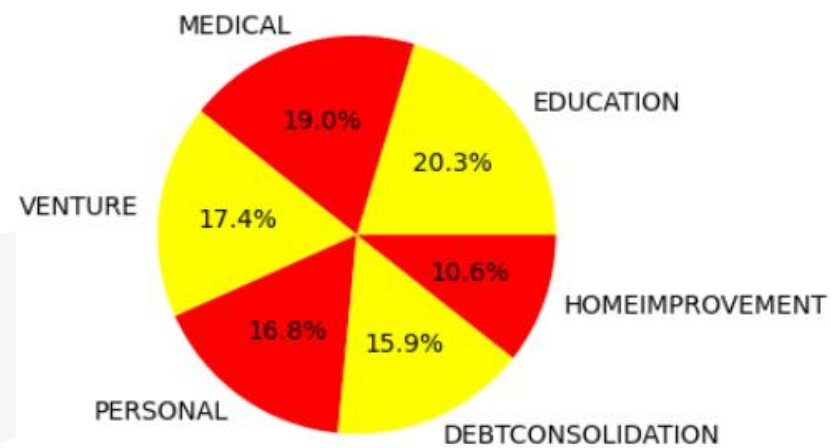
Distribution of Education



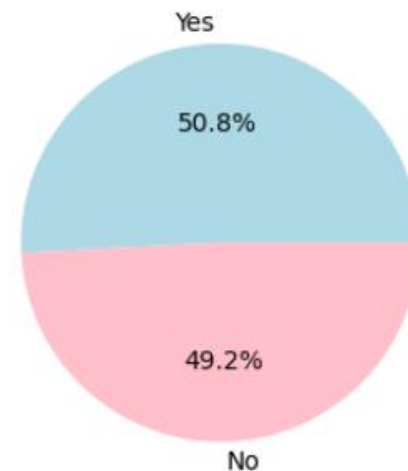
Distribution of Home Ownership



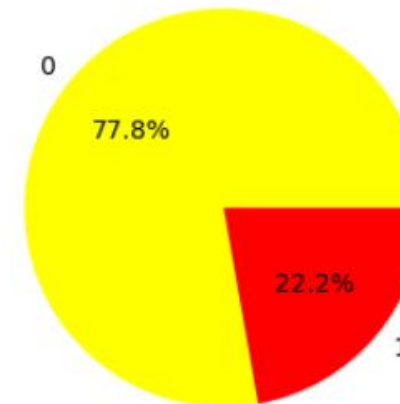
Distribution of Purpose Loan



Distribution of Indicator Previous Loan



Distribution of Loan Status



```
#Visualisasi distribusi data dengan pie chart
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 3, figsize=(12, 10)) #Membuat subplot (baris, kolom, ukuran)

gender_counts = data['person_gender'].value_counts()
axes[0,0].pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%', colors=['lightblue', 'pink'])
axes[0,0].set_title("Distribution of Gender")

education_counts = data['person_education'].value_counts()
axes[0,1].pie(education_counts, labels=education_counts.index, autopct='%1.1f%%', colors=['yellow', 'red'])
axes[0,1].set_title("Distribution of Education")

home_counts = data['person_home_ownership'].value_counts()
axes[0,2].pie(home_counts, labels=home_counts.index, autopct='%1.1f%%', colors=['lightblue', 'pink'])
axes[0,2].set_title("Distribution of Home Ownership")

intent_counts = data['loan_intent'].value_counts()
axes[1,0].pie(intent_counts, labels=intent_counts.index, autopct='%1.1f%%', colors=['yellow', 'red'])
axes[1,0].set_title("Distribution of Purpose Loan")

prev_counts = data['previous_loan_defaults_on_file'].value_counts()
axes[1,1].pie(prev_counts, labels=prev_counts.index, autopct='%1.1f%%', colors=['lightblue', 'pink'])
axes[1,1].set_title("Distribution of Indicator Previous Loan")

status_counts = data['loan_status'].value_counts()
axes[1,2].pie(status_counts, labels=status_counts.index, autopct='%1.1f%%', colors=['yellow', 'red'])
axes[1,2].set_title("Distribution of Loan Status")

plt.tight_layout()
plt.show()
```

EXPLORATORY DATA ANALYSIS (EDA)

Mapping, deskripsi data, dan missing value

```
#Merubah format data yang berbentuk objek

mapping_gender = {'male': 0, 'female': 1}
mapping_edu = {'High School': 0, 'Associate': 1, 'Bachelor': 2, 'Master':3, 'Doctorate':4} #Ordinal=>harus sesuai urutan
mapping_home = {'RENT': 0, 'MORTGAGE': 1, 'OWN': 2, 'OTHER':3}
mapping_intent = {'EDUCATION': 0, 'MEDICAL': 1, 'VENTURE': 2, 'PERSONAL':3, 'DEBTCONSOLIDATION':4, 'HOMEIMPROVEMENT':5}
mapping_previous = {'No': 0, 'Yes': 1}

data[['person_gender','person_education','person_home_ownership','loan_intent','previous_loan_defaults_on_file']] = data[['person_gender',
'person_education','person_home_ownership','loan_intent','previous_loan_defaults_on_file']].replace({
    'person_gender': mapping_gender,'person_education': mapping_edu,
    'person_home_ownership': mapping_home,'loan_intent': mapping_intent,'previous_loan_defaults_on_file': mapping_previous})
data
```

Penggunaan mapping ditujukan untuk merubah format variabel tipe objek menjadi integer sehingga nantinya dapat memudahkan analisis model.

```
#Analisis deskriptif
data.describe()
```

	person_age	person_gender	person_education	person_income	person_emp_exp	person_home_ownership	loan_amnt	loan_intent	loan_int_rate	loan_percent_income	cb_person_cred_hist_length
count	45000.000000	45000.000000	45000.000000	4.500000e+04	45000.000000	45000.000000	45000.000000	45000.000000	45000.000000	45000.000000	45000.000000
mean	27.764178	0.447978	1.383333	8.031905e+04	5.410333	0.549822	9583.157556	2.207489	11.006606	0.139725	5.867489
std	6.045108	0.497292	1.077741	8.042250e+04	6.063532	0.627919	6314.886691	1.649272	2.978808	0.087212	3.879702
min	20.000000	0.000000	0.000000	8.000000e+03	0.000000	0.000000	500.000000	0.000000	5.420000	0.000000	2.000000
25%	24.000000	0.000000	0.000000	4.720400e+04	1.000000	0.000000	5000.000000	1.000000	8.590000	0.070000	3.000000
50%	26.000000	0.000000	1.000000	6.704800e+04	4.000000	0.000000	8000.000000	2.000000	11.010000	0.120000	4.000000
75%	30.000000	1.000000	2.000000	9.578925e+04	8.000000	1.000000	12237.250000	4.000000	12.990000	0.190000	8.000000
max	144.000000	1.000000	4.000000	7.200766e+06	125.000000	3.000000	35000.000000	5.000000	20.000000	0.660000	30.000000

Analisis deskriptif mencakup informasi nilai mean, standar deviasi, nilai min. dan maks., dan nilai modus tiap variabel.

```
#Melihat apakah terdapat missing value
data.isnull().sum()
```

	0
person_age	0
person_gender	0
person_education	0
person_income	0
person_emp_exp	0
person_home_ownership	0
loan_amnt	0
loan_intent	0
loan_int_rate	0
loan_percent_income	0
cb_person_cred_hist_length	0
credit_score	0
previous_loan_defaults_on_file	0
loan_status	0

Mencari missing value agar menghindari bias dalam analisis

SPLIT DATA

Penggunaan SMOTE ditujukan ketika terjadi ketidakseimbangan pada nilai kelas. SMOTE akan mencegah terjadinya bias pada model karena akan cenderung selalu belajar memprediksi kelas yang mayoritas. Akibatnya, model sebenarnya gagal mengenali kelas minoritas meski menghasilkan nilai akurasi yang tinggi.

```
#Membagi data X dan y
from sklearn.model_selection import train_test_split
X = data.drop('loan_status', axis=1) #Hanya mengambil data selain data kelas
y = data['loan_status'] #Kolom 'loan_status' merupakan data kelas
#membagi data latih dan data uji
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

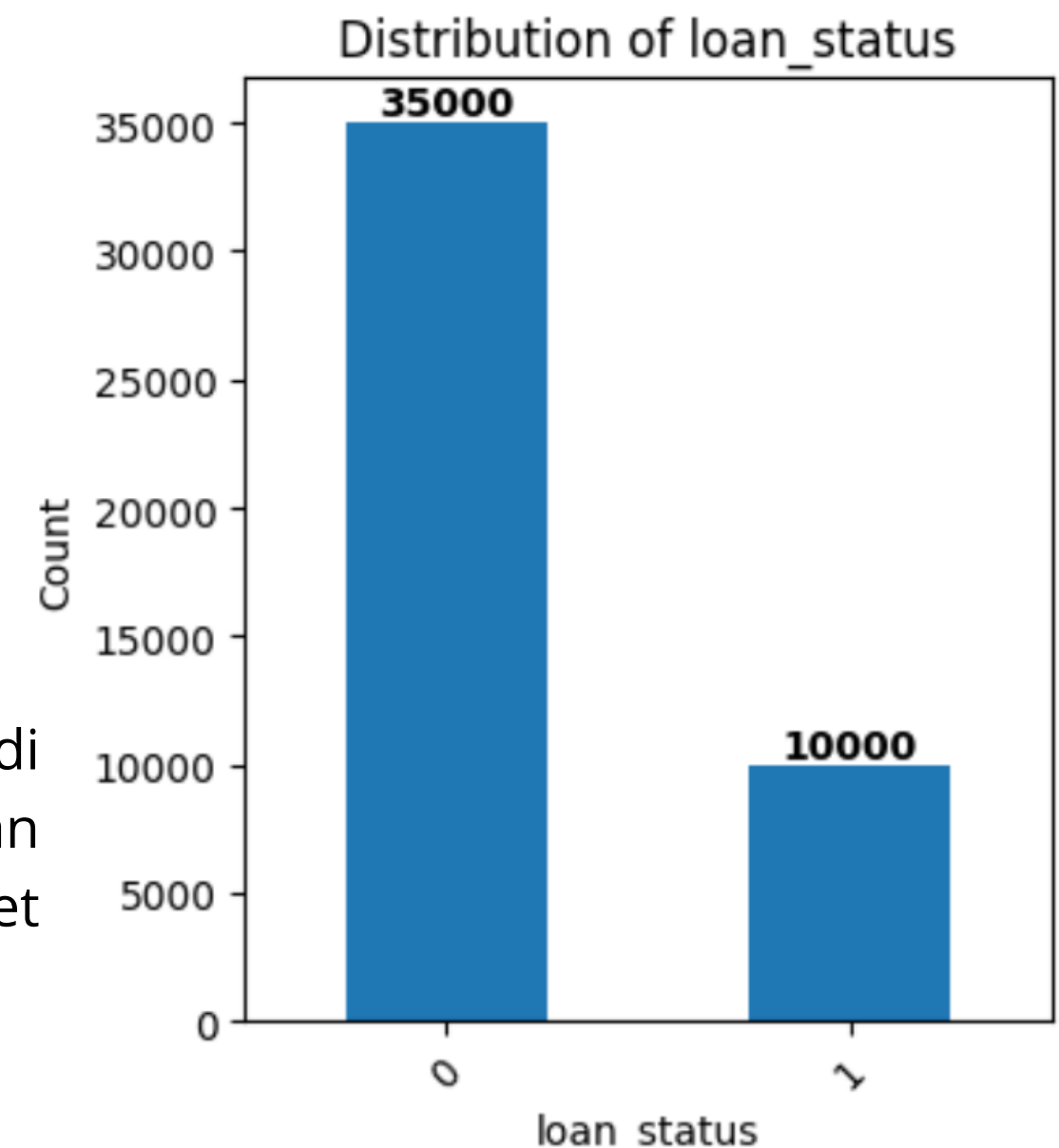
```
from imblearn.over_sampling import SMOTE
from collections import Counter

#Cek distribusi kelas sebelum SMOTE
print("Distribusi sebelum SMOTE:", Counter(y_train))
#Inisialisasi SMOTE
smote = SMOTE(sampling_strategy='auto', random_state=42)
#Menerapkan SMOTE hanya pada data latih
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
#Cek distribusi kelas setelah SMOTE
print("Distribusi setelah SMOTE:", Counter(y_train_resampled))
```

```
Distribusi sebelum SMOTE: Counter({0: 28000, 1: 8000})
Distribusi setelah SMOTE: Counter({0: 28000, 1: 28000})
```

```
#Membagi data X dan y
from sklearn.model_selection import train_test_split
X = data.drop('loan_status', axis=1) #Hanya mengambil data selain data kelas
y = data['loan_status'] #Kolom 'loan_status' merupakan data kelas
#membagi data latih dan data uji
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Membagi data X dan y di mana y adalah kelas/target. Kemudian membagi data menjadi 2 yakni data latih dan data uji.



Terjadi ketidakseimbangan nilai kelas/target

ANALISIS MODEL

01 Logistic Regression

```
#Melatih model
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train_resampled, y_train_resampled)
```

/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:46
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

LogisticRegression
LogisticRegression()

```
#Prediksi dan evaluasi model
import seaborn as sns
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
y_test_pred = model.predict(X_test)

print("Akurasi Data Uji:", accuracy_score(y_test, y_test_pred))
print("\nClassification Report1:\n", classification_report(y_test, y_test_pred))

def evaluation(test, pred):
    cm=confusion_matrix(pred, test)
    sns.heatmap(cm, annot=True, fmt='d')

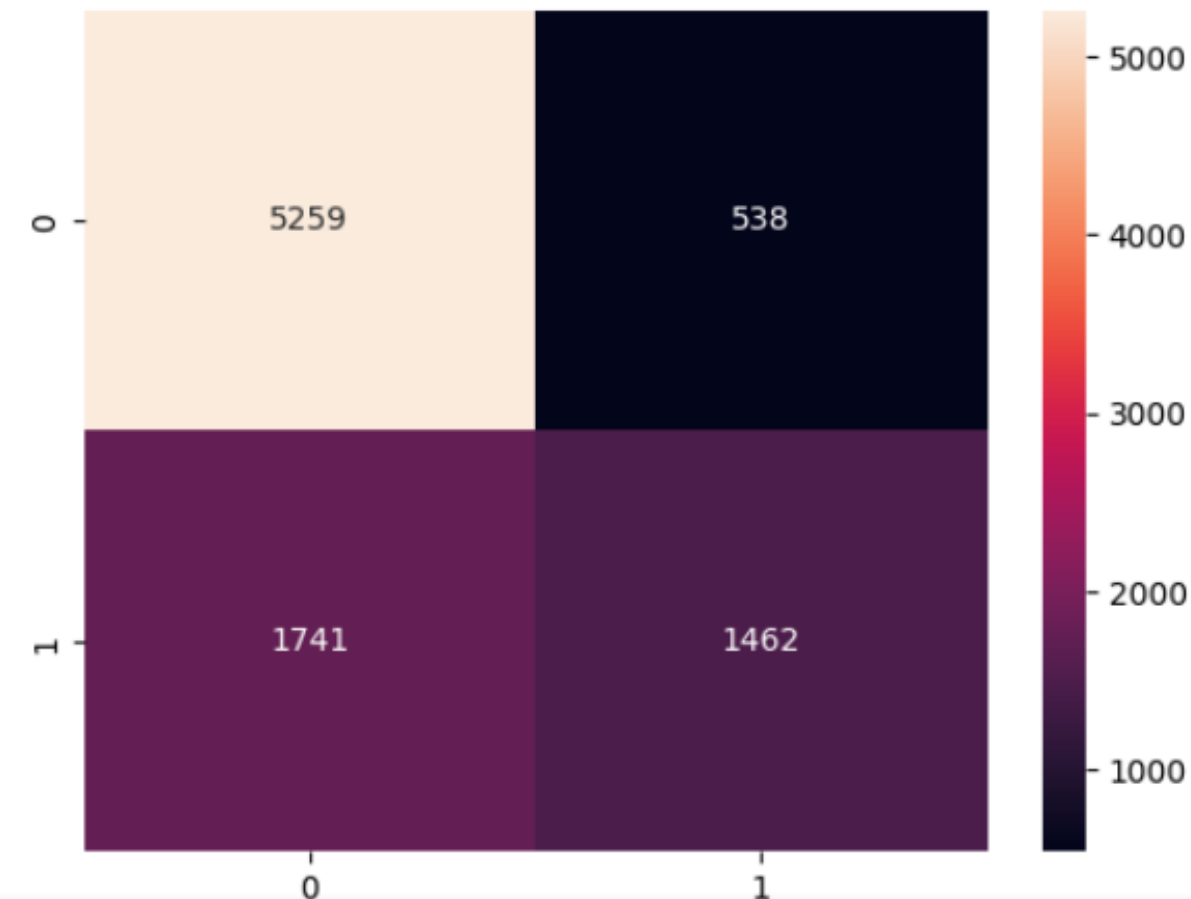
evaluation(y_test, y_test_pred)
```

Akurasi Data Uji: 0.7467777777777778

Classification Report1:

	precision	recall	f1-score	support
0	0.91	0.75	0.82	7000
1	0.46	0.73	0.56	2000
accuracy			0.75	9000
macro avg	0.68	0.74	0.69	9000
weighted avg	0.81	0.75	0.76	9000

Classification report



Confusion Matrix

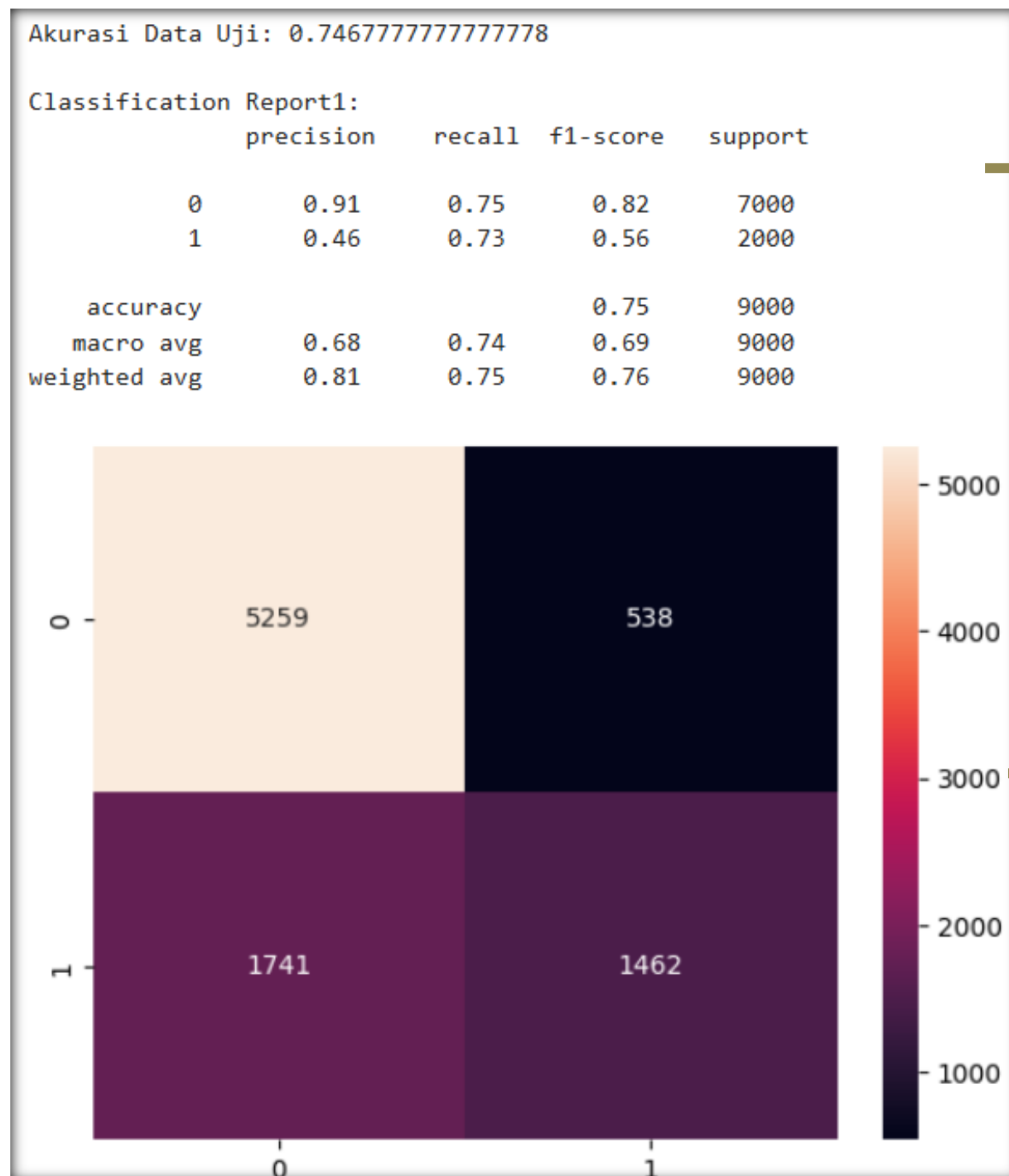
Akurasi = 74,68%

True Negative (TN) = 5259; model memprediksi kelas 0 dan benar data asli di kelas 0.

False Negative (FN) = 538; model memprediksi kelas 0, tetapi data asli di kelas 1.

False Positive (FP) = 1741; Model memprediksi kelas 1, tetapi data asli di kelas 0.

True Positive (TP) = 1462; model memprediksi kelas 1 dan benar data asli di kelas 1



Classification
report

Confusion Matrix

02 Decision Tree

```
#Melatih model
from sklearn.tree import DecisionTreeClassifier, plot_tree
model_tree = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=0)
model_tree.fit(X_train_resampled, y_train_resampled)
```

DecisionTreeClassifier

DecisionTreeClassifier(max_depth=5, random_state=0)

```
#Prediksi dan evaluasi model
import seaborn as sns
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
y_test_pred = model.predict(X_test)
print("Akurasi Data Uji:", accuracy_score(y_test, y_test_pred))
print("\nClassification Report1:\n", classification_report(y_test, y_test_pred))

def evaluation(test, pred):
    cm=confusion_matrix(pred, test)
    sns.heatmap(cm, annot=True, fmt='d')

evaluation(y_test, y_test_pred)
```

Akurasi = 74,68%

True Negative (TN) = 5259; model memprediksi kelas 0 dan benar data asli di kelas 0.

False Negative (FN) = 538; model memprediksi kelas 0, tetapi data asli di kelas 1.

False Positive (FP) = 1741; Model memprediksi kelas 1, tetapi data asli di kelas 0.

True Positive (TP) = 1462; model memprediksi kelas 1 dan benar data asli di kelas 1

```
#Melatih model
from sklearn.svm import SVC

# Macam kernel: 'rbf', 'linear', 'poly', 'sigmoid'
model = SVC(kernel='rbf', C=1.0, gamma='scale', random_state=42)
model.fit(X_train_resampled, y_train_resampled)
```

SVC

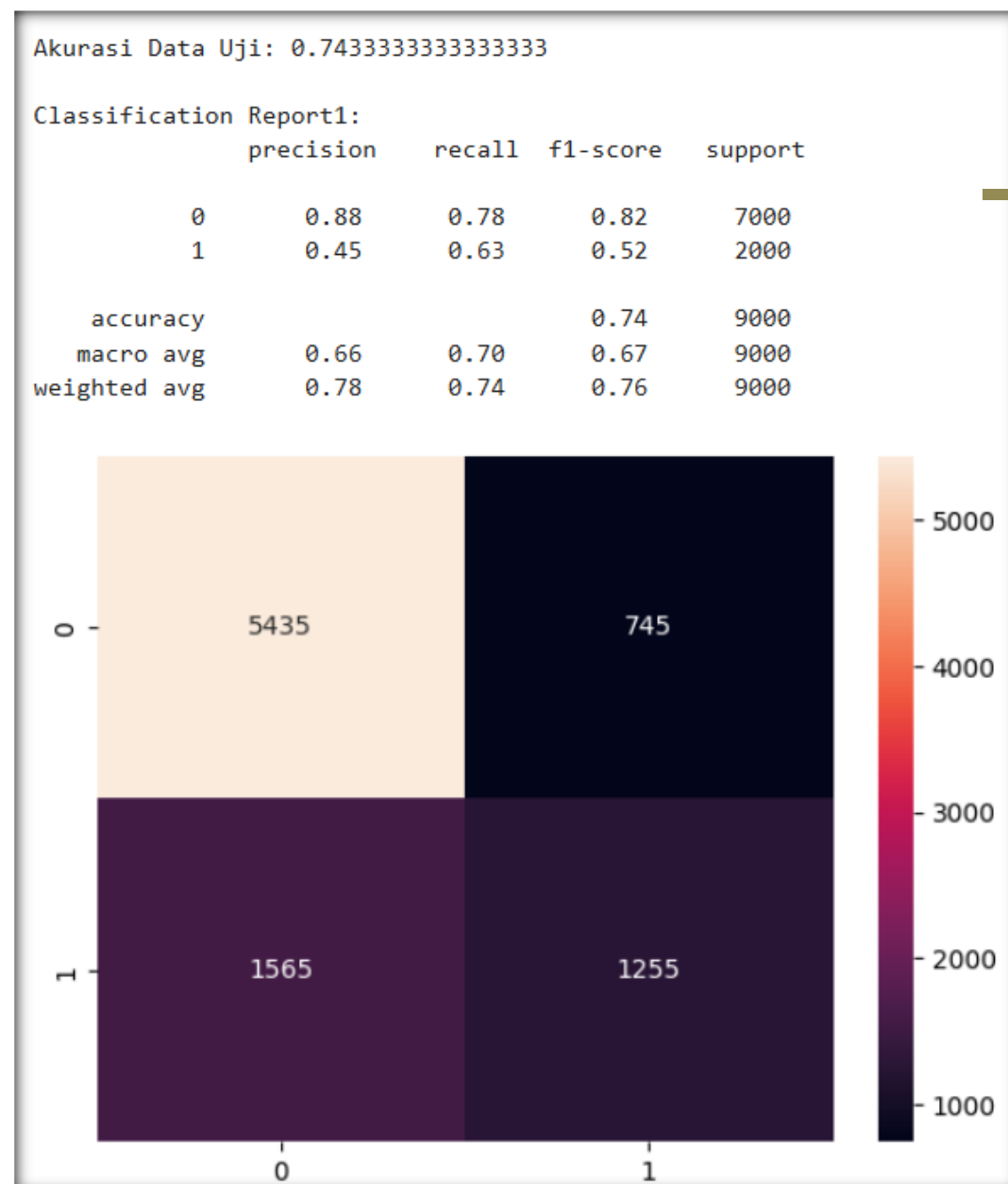
SVC(random_state=42)

```
import seaborn as sns
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
y_test_pred = model.predict(X_test)

print("Akurasi Data Uji:", accuracy_score(y_test, y_test_pred))
print("\nClassification Report1:\n", classification_report(y_test, y_test_pred))

def evaluation(test, pred):
    cm=confusion_matrix(pred, test)
    sns.heatmap(cm, annot=True, fmt='d')

evaluation(y_test, y_test_pred)
```



Classification
report

Confusion Matrix

Akurasi = 74,33%

True Negative (TN) = 5435; model memprediksi kelas 0 dan benar data asli di kelas 0.

False Negative (FN) = 745; model memprediksi kelas 0, tetapi data asli di kelas 1.

False Positive (FP) = 1565; Model memprediksi kelas 1, tetapi data asli di kelas 0.

True Positive (TP) = 1255; model memprediksi kelas 1 dan benar data asli di kelas 1

04

Random Forest

```
#Melatih model
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, criterion='entropy', max_depth=None, random_state=42)
model.fit(X_train_resampled, y_train_resampled)
```

```
RandomForestClassifier
RandomForestClassifier(criterion='entropy', random_state=42)
```

```
#Prediksi dan evaluasi model
import seaborn as sns
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
y_test_pred = model.predict(X_test)

print("Akurasi Data Uji:", accuracy_score(y_test, y_test_pred))
print("\nClassification Report1:\n", classification_report(y_test, y_test_pred))

def evaluation(test, pred):
    cm = confusion_matrix(pred, test)
    sns.heatmap(cm, annot=True, fmt='d')

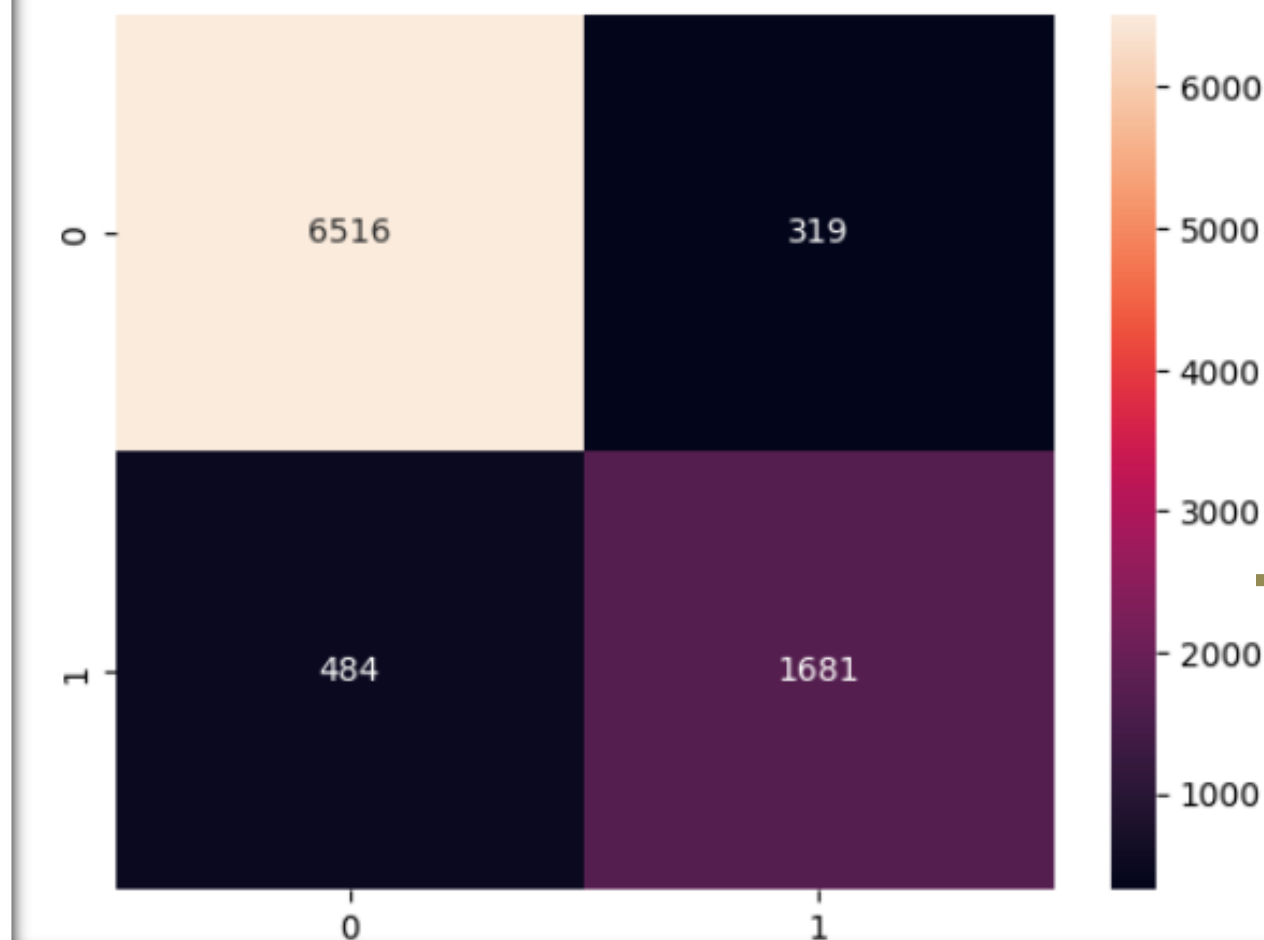
evaluation(y_test, y_test_pred)
```

Akurasi Data Uji: 0.9107777777777778

```
Classification Report1:
              precision    recall  f1-score   support

     0       0.95       0.93       0.94       7000
     1       0.78       0.84       0.81       2000

 accuracy          0.86       0.89       0.91       9000
 macro avg       0.86       0.89       0.87       9000
 weighted avg    0.91       0.91       0.91       9000
```



Classification
report

Confusion Matrix

Akurasi = 91,08%

True Negative (TN) = 6516; model memprediksi kelas 0 dan benar data asli di kelas 0.

False Negative (FN) = 319; model memprediksi kelas 0, tetapi data asli di kelas 1.

False Positive (FP) = 484; Model memprediksi kelas 1, tetapi data asli di kelas 0.

True Positive (TP) = 1681; model memprediksi kelas 1 dan benar data asli di kelas 1

04 Naïve Bayes

```
#Melatih model
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train_resampled, y_train_resampled)
```

→ GaussianNB

GaussianNB()

```
#Prediksi dan evaluasi model
import seaborn as sns
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
y_test_pred = model.predict(X_test)

print("Akurasi Data Uji:", accuracy_score(y_test, y_test_pred))
print("\nClassification Report1:\n", classification_report(y_test, y_test_pred))

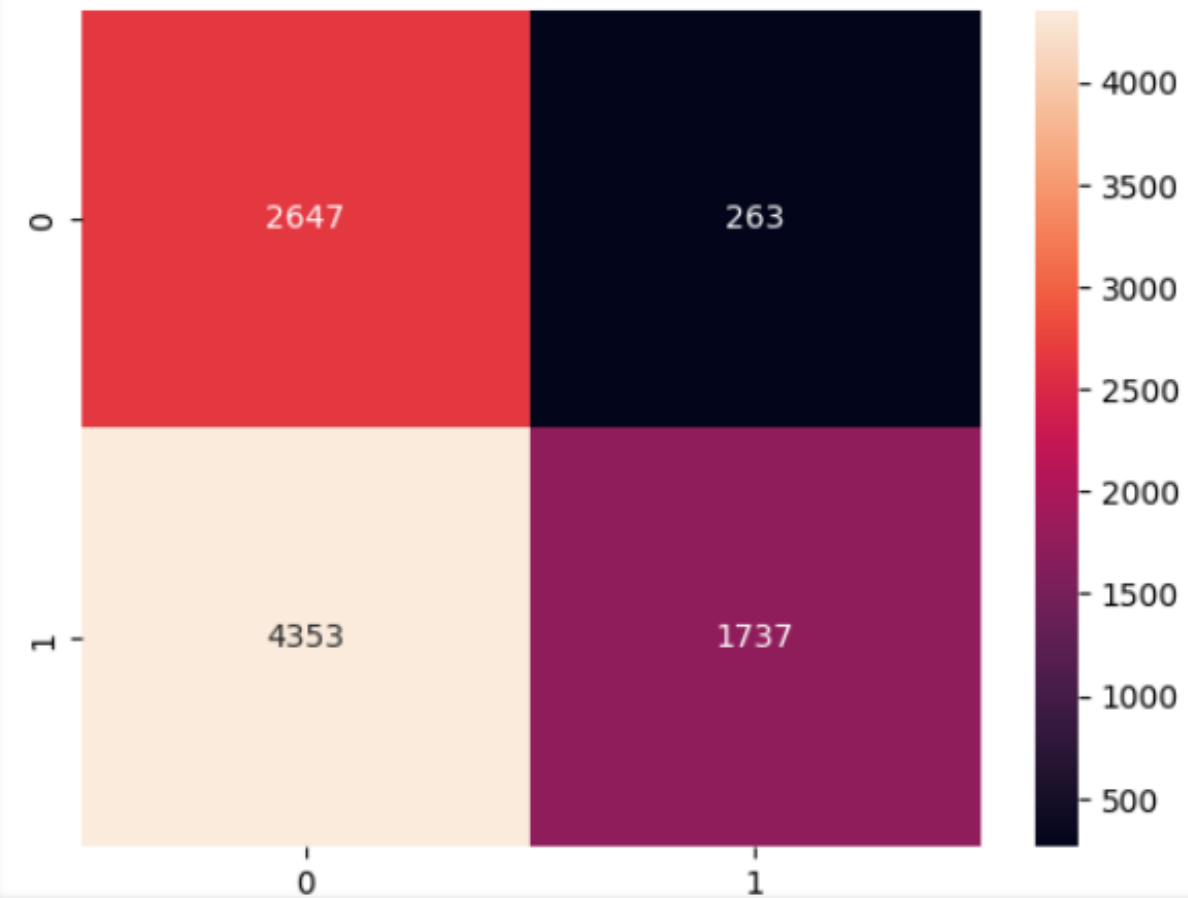
def evaluation(test, pred):
    cm = confusion_matrix(pred, test)
    sns.heatmap(cm, annot=True, fmt='d')

evaluation(y_test, y_test_pred)
```

Akurasi Data Uji: 0.4871111111111111

Classification Report1:

	precision	recall	f1-score	support
0	0.91	0.38	0.53	7000
1	0.29	0.87	0.43	2000
accuracy			0.49	9000
macro avg	0.60	0.62	0.48	9000
weighted avg	0.77	0.49	0.51	9000



Classification
report

Confusion Matrix

Akurasi = 48,71%

True Negative (TN) = 2647; model memprediksi kelas 0 dan benar data asli di kelas 0.

False Negative (FN) = 263; model memprediksi kelas 0, tetapi data asli di kelas 1.

False Positive (FP) = 4353; Model memprediksi kelas 1, tetapi data asli di kelas 0.

True Positive (TP) = 1737; model memprediksi kelas 1 dan benar data asli di kelas 1

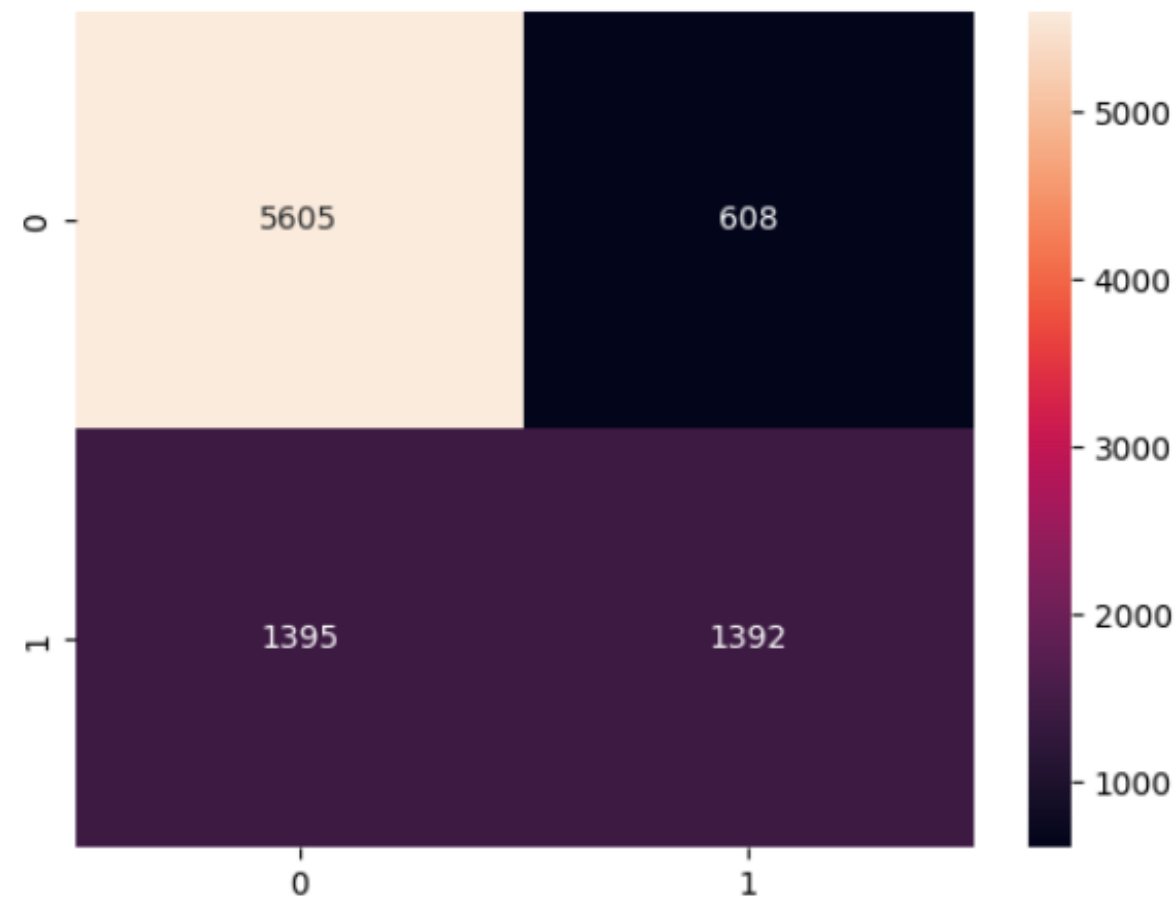
05

K-Nearest Neighbor

Akurasi Data Uji: 0.7774444444444445

Classification Report1:

	precision	recall	f1-score	support
0	0.90	0.80	0.85	7000
1	0.50	0.70	0.58	2000
accuracy			0.78	9000
macro avg	0.70	0.75	0.71	9000
weighted avg	0.81	0.78	0.79	9000

Classification
report

Confusion Matrix

```
#Melatih model
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5, weights='distance', algorithm='kd_tree', metric='minkowski')
model.fit(X_train_resampled, y_train_resampled)
```

```
KNeighborsClassifier
KNeighborsClassifier(algorithm='kd_tree', weights='distance')
```

```
#Prediksi dan evaluasi model
import seaborn as sns
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
y_test_pred = model.predict(X_test)

print("Akurasi Data Uji:", accuracy_score(y_test, y_test_pred))
print("\nClassification Report1:\n", classification_report(y_test, y_test_pred))

def evaluation(test, pred):
    cm = confusion_matrix(pred, test)
    sns.heatmap(cm, annot=True, fmt='d')

evaluation(y_test, y_test_pred)
```

Akurasi = 77,74%

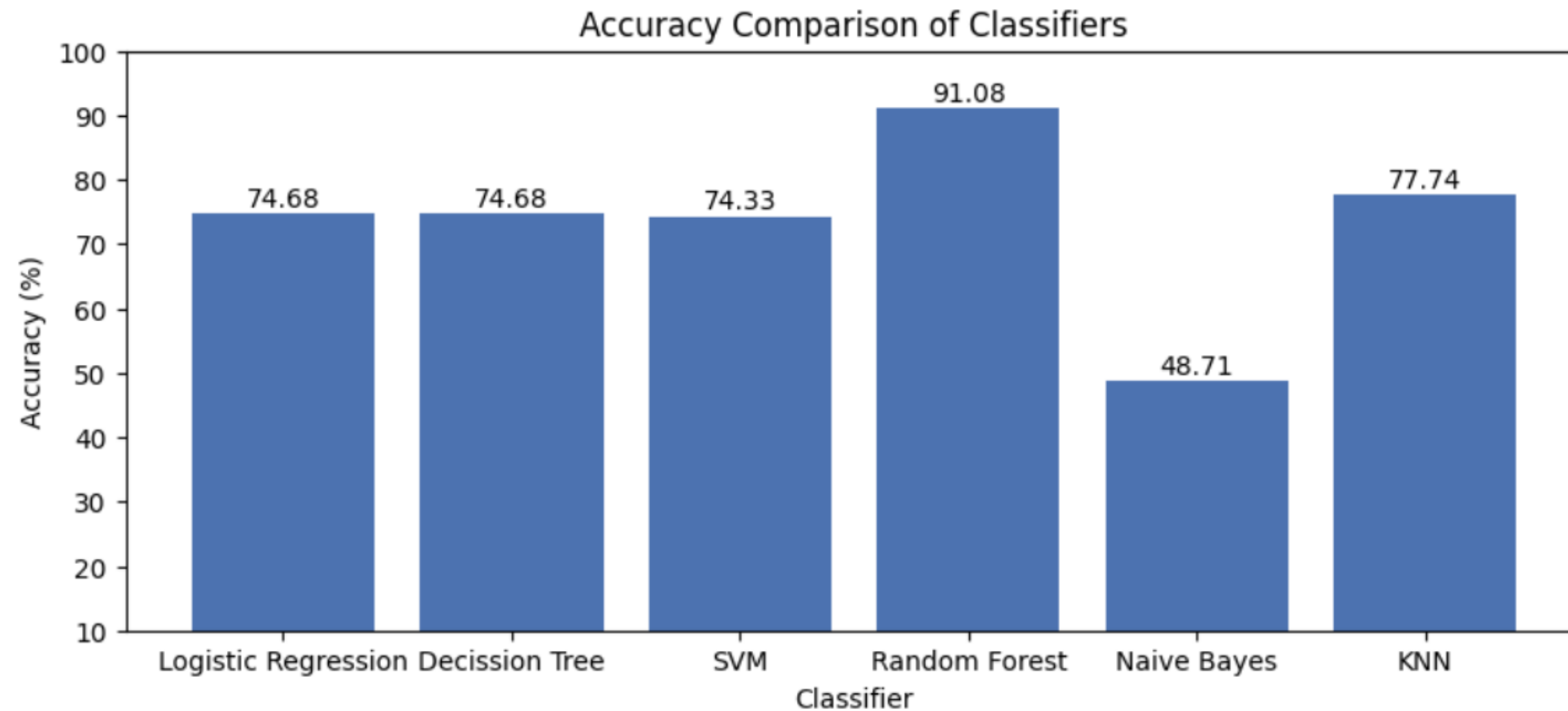
True Negative (TN) = 2647; model memprediksi kelas 0 dan benar data asli di kelas 0.

False Negative (FN) = 263; model memprediksi kelas 0, tetapi data asli di kelas 1.

False Positive (FP) = 4353; Model memprediksi kelas 1, tetapi data asli di kelas 0.

True Positive (TP) = 1737; model memprediksi kelas 1 dan benar data asli di kelas 1

PERBANDINGAN AKURASI



Dapat dilihat dari keenam model, Random Forest memiliki nilai akurasi tertinggi yang mencapai 91,08% dan nilai terendah ada pada Naïve Bayes yang mencapai 48,71%. Untuk model yang lain mencapai akurasi 70% ke atas.